Stacked Imbalance Zones [BOSWaves

```
// This Pine Script® code is subject to the terms of the Mozilla Public License 2.0 at https://
mozilla.org/MPL/2.0/
// © BOSWaves

//@version=6
indicator("Stacked Imbalance Zones [BOSWaves]", overlay = true, max_boxes_count = 500,
max_lines_count = 500, max_labels_count = 500)

// ┌──────────────────────────────────────────────────────── BOSWaves — Constants
────────────────────────────────────────────────────────┐
//
└──────────────────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────┘

const string GRP_IMBAL  = "Imbalance Detection"
const string GRP_ZONE   = "Zone Construction"
const string GRP_DELTA  = "Delta Ribbon"
const string GRP_VISUAL = "Visualization"

color GREEN = #00ff00
color RED   = #ff0000

const string TT_THRESH = "Dominance threshold. Buy-dominant when (close-low)/(high-low)
exceeds this. Sell-dominant when (high-close)/(high-low) exceeds it."
const string TT_STACK  = "Minimum consecutive dominant bars to form a stacked imbalance zone."
const string TT_VFILT  = "Volume must exceed SMA x this factor for a bar to qualify."
const string TT_VLKB   = "SMA lookback for the volume filter baseline."
const string TT_MITG   = "How zones are invalidated. Absolute = close beyond full zone. Middle =
close past midpoint."
const string TT_MAXZ   = "Maximum zones per side. Oldest removed first."
const string TT_OVER   = "Remove older overlapping same-direction zones."

// ┌──────────────────────────────────────────────────────── BOSWaves — User Inputs
────────────────────────────────────────────────────────┐
//
└──────────────────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────┘
```

```
bool    i_show        = input.bool(true, "Show Zones", group = GRP_IMBAL)
float   i_thresh      = input.float(0.62, "Dominance Threshold", step = 0.02, group = GRP_IMBAL,
tooltip = TT_THRESH)
int     i_minStack    = input.int(1, "Min Stack Count", group = GRP_IMBAL, tooltip = TT_STACK)
float   i_volMult     = input.float(0.7, "Volume Filter (×SMA)", minval = 0.1, step = 0.1, group =
GRP_IMBAL, tooltip = TT_VFILT)
int     i_volLen      = input.int(66, "Volume SMA Length", minval = 5, maxval = 100, group =
GRP_IMBAL, tooltip = TT_VLKB)

int     i_maxZones    = input.int(4, "Zones Per Side", minval = 1, maxval = 10, group = GRP_ZONE,
tooltip = TT_MAXZ)
string  i_mitigation  = input.string("Absolute", "Mitigation", options = ["Absolute", "Middle"], group
= GRP_ZONE, tooltip = TT_MITG)
bool    i_hideOverlap = input.bool(true, "Hide Overlap", group = GRP_ZONE, tooltip = TT_OVER)
int     i_extend      = input.int(40, "Zone Extension", minval = 10, maxval = 500, step = 10, group =
GRP_ZONE,
    tooltip = "How far zones project to the right in bars.")

bool    i_showDelta   = input.bool(true, "Show Delta Ribbon", group = GRP_DELTA)
float   i_deltaScale  = input.float(0.4, "Ribbon Height", minval = 0.1, maxval = 1.0, step = 0.05, group
= GRP_DELTA)
float   i_deltaSmooth = input.float(0.92, "Ribbon Smoothing", minval = 0.5, maxval = 0.99, step =
0.01, group = GRP_DELTA)
float   i_deltaReact  = input.float(0.35, "Ribbon Reactivity", minval = 0.05, maxval = 1.0, step = 0.05,
group = GRP_DELTA)
int     i_deltaThin   = input.int(1, "Ribbon Density", minval = 1, maxval = 3, group = GRP_DELTA)
int     i_deltaWidth  = input.int(5, "Ribbon Thickness", minval = 2, maxval = 8, group = GRP_DELTA)
bool    i_showEdge    = input.bool(true, "Ribbon Edge Line", group = GRP_DELTA)

bool    i_showLabel   = input.bool(true, "Show Metrics", group = GRP_VISUAL)
color   i_bullCss     = input.color(#00ff00, "Demand Zone", group = GRP_VISUAL, inline = "zc")
color   i_bearCss     = input.color(#ff0000, "Supply Zone", group = GRP_VISUAL, inline = "zc")
int     i_gradientRows = input.int(4, "Gradient Layers", minval = 2, maxval = 6, group = GRP_VISUAL,
    tooltip = "Number of gradient layers inside the zone. More = smoother fade.")

// ┌──────────────────────────────────────────── BOSWaves — Helpers
──────────────────────────────────────────────────────────────────┐
//
```

```
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

f_vol_text(float v) =>
    v >= 1000000000 ? str.tostring(math.round(v / 1000000000, 3)) + "B" :
     v >= 1000000   ? str.tostring(math.round(v / 1000000, 3)) + "M" :
     v >= 1000      ? str.tostring(math.round(v / 1000, 3)) + "K" :
     str.tostring(math.round(v))

// ┌──────────────────────────────────────────────── BOSWaves — Per-Bar Flow Scan ────────────────────────────────────────────────┐
//
 └──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

float bar_rng   = high - low
float buy_ratio = bar_rng > 0 ? (close - low) / bar_rng : 0.5
float sel_ratio = 1.0 - buy_ratio
float vol_sma   = ta×sma(volume, i_volLen)
bool  vol_ok    = volume >= vol_sma * i_volMult
bool  is_buy    = buy_ratio >= i_thresh and vol_ok
bool  is_sell   = sel_ratio >= i_thresh and vol_ok

float bar_delta  = volume * (buy_ratio - sel_ratio)
float abs_delta  = math×abs(bar_delta)
float delta_sma  = ta.sma(abs_delta, 50)
float norm_delta = delta_sma > 0 ? math.max(-1., math.min(1., bar_delta / (delta_sma * 2.))) : 0.

// ┌──────────────────────────────────────────────── BOSWaves — Zone Data Structure ────────────────────────────────────────────────┐
//
 └──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

type StackZone
    float   top
    float   btm
    float   avg
    float   buy_pct
    float   sell_pct
    float   total_vol
    float   relevance
```

```
    int     stack_n
    int     left_time
    int     left_bi
    bool    is_bull
    float   ribbon_val
    float   prev_ribbon_y

type StackDraw
    box     ob
    box     eOB
    line    mL
    array<box>  gradBoxes
    line    accentTop
    line    accentBot
    line    accentTopExt
    line    accentBotExt
    array<line> ribbonFill
    array<line> ribbonEdge

var StackZone[] bull_zones = array.new<StackZone>()
var StackZone[] bear_zones = array.new<StackZone>()
var StackDraw[] bull_draws = array.new<StackDraw>()
var StackDraw[] bear_draws = array.new<StackDraw>()

// ┌──────────────────────────── BOSWaves — Stacked Imbalance Run Tracking
────────────────────────────────────────────┐
//
 └──────────────────────────────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────┘

var int   bull_run      = 0
var int   bear_run      = 0
var float bull_zt       = na
var float bull_zb       = na
var float bear_zt       = na
var float bear_zb       = na
var int   bull_start_t  = na
var int   bear_start_t  = na
var float bull_vol_sum  = 0.0
```

```
var float bear_vol_sum  = 0.0
var float bull_buy_sum  = 0.0
var float bear_sell_sum = 0.0

// ┌──────────────────────────── BOSWaves — Delete Draw Helper ────────────────────────────┐
//
└──────────────────────────────────────────────────────────────────────────────────────────┘

f_delete_draw(StackDraw d) =>
    box.delete(d.ob)
    box.delete(d.eOB)
    line.delete(d.mL)
    for b in d.gradBoxes
        b.delete()
    d.gradBoxes.clear()
    line.delete(d.accentTop)
    line.delete(d.accentBot)
    line.delete(d.accentTopExt)
    line.delete(d.accentBotExt)
    for ln in d.ribbonFill
        ln.delete()
    d.ribbonFill.clear()
    for ln in d.ribbonEdge
        ln.delete()
    d.ribbonEdge.clear()

// ┌──────────────────────────── BOSWaves — Detect Run Breaks & Fire Zones ────────────────────────────┐
//
└──────────────────────────────────────────────────────────────────────────────────────────┘

bool fire_bull = false
bool fire_bear = false

float  snap_bull_zt  = bull_zt
float  snap_bull_zb  = bull_zb
int    snap_bull_cnt = bull_run
```

```
float  snap_bull_vol = bull_vol_sum
float  snap_bull_buy = bull_run > 0 ? bull_buy_sum / bull_run : 0.5
int    snap_bull_t   = bull_start_t

float  snap_bear_zt  = bear_zt
float  snap_bear_zb  = bear_zb
int    snap_bear_cnt = bear_run
float  snap_bear_vol = bear_vol_sum
float  snap_bear_sel = bear_run > 0 ? bear_sell_sum / bear_run : 0.5
int    snap_bear_t   = bear_start_t

if bull_run >= i_minStack and not is_buy
    fire_bull := true
if bear_run >= i_minStack and not is_sell
    fire_bear := true
if is_buy and bear_run >= i_minStack
    fire_bear := true
if is_sell and bull_run >= i_minStack
    fire_bull := true

//  ┌──────────────────────────── BOSWaves — Create Draw Objects Helper
──────────────────────────────────────────────────────────────┐
//
 └──────────────────────────────────────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────┘

f_create_draw(color css) =>
    box _ob  = box.new(na, na, na, na, xloc = xloc.bar_time, border_color = color×new(css, 60),
bgcolor = color.new(color.black, 100), border_width = 0)
    box _eOB = box.new(na, na, na, na, xloc = xloc.bar_time, border_color = color×new(css, 60),
bgcolor = color.new(color.black, 100), border_width = 0)
    line _mL = line.new(na, na, na, na, xloc = xloc.bar_time, color = color×new(css, 60), width = 1,
style = line.style_dashed)

    gradB = array.new<box>()
    for i = 0 to i_gradientRows - 1
        gradB.push(box.new(na, na, na, na, xloc = xloc.bar_time, border_color = color.new(color.black,
100), bgcolor = na, border_width = 0))
```

```
    line aT  = line.new(na, na, na, na, xloc = xloc.bar_time, color = na, width = 2)
    line aB  = line.new(na, na, na, na, xloc = xloc.bar_time, color = na, width = 2)
    line aTE = line.new(na, na, na, na, xloc = xloc.bar_time, color = na, width = 2)
    line aBE = line.new(na, na, na, na, xloc = xloc.bar_time, color = na, width = 2)

    StackDraw.new(_ob, _eOB, _mL, gradB, aT, aB, aTE, aBE, array.new<line>(), array.new<line>())

// ┌──────────────────────────── BOSWaves — Push New Zone on Fire
──────────────────────────────────────────────────────────────────────┐
//
 └──────────────────────────────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────────┘
if fire_bull and not na(snap_bull_zt) and not na(snap_bull_zb) and i_show and snap_bull_zt >
snap_bull_zb
    float _avg = math×avg(snap_bull_zt, snap_bull_zb)
    float _bp  = snap_bull_buy * 100.0
    float _sp  = 100.0 - _bp

    if i_hideOverlap and bull_zones.size() > 0
        for k = bull_zones.size() - 1 to 0
            StackZone ez = bull_zones.get(k)
            if snap_bull_zb < ez.top and snap_bull_zt > ez.btm
                f_delete_draw(bull_draws.get(k))
                bull_zones.remove(k)
                bull_draws.remove(k)
                break

    bull_zones.unshift(StackZone.new(snap_bull_zt, snap_bull_zb, _avg, _bp, _sp,
        snap_bull_vol, 0.0, snap_bull_cnt, snap_bull_t, bar_index, true, 0., _avg))
    bull_draws.unshift(f_create_draw(i_bullCss))

if fire_bear and not na(snap_bear_zt) and not na(snap_bear_zb) and i_show and snap_bear_zt >
snap_bear_zb
    float _avg = math×avg(snap_bear_zt, snap_bear_zb)
    float _sp  = snap_bear_sel * 100.0
    float _bp  = 100.0 - _sp

    if i_hideOverlap and bear_zones.size() > 0
```

```
        for k = bear_zones.size() - 1 to 0
            StackZone ez = bear_zones.get(k)
            if snap_bear_zb < ez.top and snap_bear_zt > ez.btm
                f_delete_draw(bear_draws.get(k))
                bear_zones.remove(k)
                bear_draws.remove(k)
                break

    bear_zones.unshift(StackZone.new(snap_bear_zt, snap_bear_zb, _avg, _bp, _sp,
         snap_bear_vol, 0.0, snap_bear_cnt, snap_bear_t, bar_index, false, 0., _avg))
    bear_draws.unshift(f_create_draw(i_bearCss))

//  ┌──────────────────────────── BOSWaves — Update Run Tracking
──────────────────────────────────────────────────────────────────────────────┐
//
 └──────────────────────────────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────────────────────┘
float bt = math.max(close, open)
float bb = math.min(close, open)

if is_buy
    if bull_run == 0
        bull_start_t  := time
        bull_zt       := bt
        bull_zb       := bb
        bull_vol_sum  := volume
        bull_buy_sum  := buy_ratio
    else
        bull_zt       := math.max(bull_zt, bt)
        bull_zb       := math.min(bull_zb, bb)
        bull_vol_sum  += volume
        bull_buy_sum  += buy_ratio
    bull_run += 1
    bear_run      := 0
    bear_vol_sum  := 0.0
    bear_sell_sum := 0.0
else if is_sell
    if bear_run == 0
```

```
        bear_start_t  := time
        bear_zt       := bt
        bear_zb       := bb
        bear_vol_sum  := volume
        bear_sell_sum := sel_ratio
    else
        bear_zt       := math.max(bear_zt, bt)
        bear_zb       := math.min(bear_zb, bb)
        bear_vol_sum  += volume
        bear_sell_sum += sel_ratio
    bear_run += 1
    bull_run      := 0
    bull_vol_sum  := 0.0
    bull_buy_sum  := 0.0
else
    bull_run := 0, bear_run := 0
    bull_vol_sum := 0.0, bear_vol_sum := 0.0
    bull_buy_sum := 0.0, bear_sell_sum := 0.0

// ┌──────────────────────────── BOSWaves — Zone Cap Enforcement
────────────────────────────────────────────────────────────────┐
//
 └──────────────────────────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────┘

while bull_zones.size() > i_maxZones
    bull_zones.pop()
    f_delete_draw(bull_draws.pop())

while bear_zones.size() > i_maxZones
    bear_zones.pop()
    f_delete_draw(bear_draws.pop())

// ┌──────────────────────────── BOSWaves — Delta Ribbon + Mitigation
──────────────────────────────────────────────────────────┐
//
 └──────────────────────────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────┘

f_process_zones(StackZone[] zArr, StackDraw[] dArr, string side) =>
```

```
    if zArr.size() > 0
        for k = zArr.size() - 1 to 0
            StackZone z = zArr×get(k)
            StackDraw d = dArr.get(k)

            int barsSince = bar_index - z.left_bi

            if i_showDelta and barsSince > 0
                z.ribbon_val := z.ribbon_val * i_deltaSmooth + norm_delta * i_deltaReact
                z.ribbon_val := math.max(-1., math.min(1., z.ribbon_val))

                float halfZone = (z.top - z.btm) * i_deltaScale
                float ribbonY  = z.avg + z.ribbon_val * halfZone

                if barsSince % i_deltaThin == 0 and bar_index < last_bar_index - 1
                    color fCol  = z×ribbon_val >= 0 ? GREEN : RED
                    float mag   = math×abs(z×ribbon_val)
                    int fTransp = int(math×max(15, 55 - mag × 40))

                    d.ribbonFill.push(line.new(bar_index, z.avg, bar_index, ribbonY,
                         color = color×new(fCol, fTransp), width = i_deltaWidth))

                    if i_showEdge and not na(z.prev_ribbon_y) and z.prev_ribbon_y != z.avg
                        color eCol  = z×ribbon_val >= 0 ? GREEN : RED
                        int eTransp = int(math×max(0, 30 - mag × 30))
                        d.ribbonEdge.push(line.new(bar_index - i_deltaThin, z.prev_ribbon_y, bar_index,
ribbonY,
                             color = color×new(eCol, eTransp), width = 1))

                    z.prev_ribbon_y := ribbonY

            if barstate.isconfirmed
                if side == "bull"
                    float tg = i_mitigation == "Middle" ? z.avg : z.btm
                    if close < tg
                        f_delete_draw(d)
                        zArr.remove(k)
                        dArr.remove(k)
```

```
            else
                float tg = i_mitigation == "Middle" ? z.avg : z.top
                if close > tg
                    f_delete_draw(d)
                    zArr.remove(k)
                    dArr.remove(k)

if i_show
    f_process_zones(bull_zones, bull_draws, "bull")
    f_process_zones(bear_zones, bear_draws, "bear")

// ┌──────────────────────── BOSWaves — Draw Zones on Last Bar ────────────────────────┐
//
└──────────────────────────────────────────────────────────────────────────────────────┘
f_draw_all(StackZone[] zArr, StackDraw[] dArr, color css, string side) =>
    if barstate.islast and zArr.size() > 0
        float tV = 0.0
        for j = 0 to zArr.size() - 1
            tV += zArr.get(j).total_vol
        for y = 0 to zArr.size() - 1
            StackZone z = zArr.get(y)
            z.relevance := tV > 0 ? math.floor((z.total_vol / tV) * 100.0) : 0.0
            zArr.set(y, z)

        int dt = time - time[1]
        int extTime = time + dt * i_extend

        for i = 0 to zArr.size() - 1
            StackZone z = zArr×get(i)
            StackDraw d = dArr.get(i)

            float zoneH = z×top - z×btm
            float rowH  = zoneH / i_gradientRows

            // ─── Gradient fill layers ───
            for g = 0 to d.gradBoxes.size() - 1
```

```
    float frac = float(g) / float(i_gradientRows)
    int transp = 0

    if side == "bull"
        transp := int(78 + frac * 17)
        float rBot = z×btm + g × rowH
        float rTop = rBot + rowH
        d.gradBoxes.get(g).set_lefttop(z.left_time, rTop)
        d.gradBoxes.get(g).set_rightbottom(extTime, rBot)
    else
        transp := int(78 + frac * 17)
        float rTop = z×top - g × rowH
        float rBot = rTop - rowH
        d.gradBoxes.get(g).set_lefttop(z.left_time, rTop)
        d.gradBoxes.get(g).set_rightbottom(extTime, rBot)

    d.gradBoxes.get(g).set_bgcolor(color.new(css, transp))

// ——— Outer shell box ———
d.ob.set_lefttop(z.left_time, z.top)
d.ob.set_rightbottom(time, z.btm)
d.ob.set_border_color(color.new(css, 70))
d.ob.set_border_width(1)
d.ob.set_bgcolor(color.new(color.black, 100))

// ——— Extended box ———
d.eOB.set_lefttop(time, z.top)
d.eOB.set_rightbottom(extTime, z.btm)
d.eOB.set_border_color(color.new(css, 80))
d.eOB.set_border_width(1)
d.eOB.set_bgcolor(color.new(color.black, 100))

// ——— Accent lines ———
color accentCol = color.new(css, 15)

if side == "bull"
    d.accentBot.set_xy1(z.left_time, z.btm)
    d.accentBot.set_xy2(time, z.btm)
```

```
            d.accentBot.set_color(accentCol)
            d.accentBotExt.set_xy1(time, z.btm)
            d.accentBotExt.set_xy2(extTime, z.btm)
            d.accentBotExt.set_color(color.new(css, 40))
            d.accentTop.set_xy1(z.left_time, z.top)
            d.accentTop.set_xy2(time, z.top)
            d.accentTop.set_color(color.new(css, 75))
            d.accentTopExt.set_xy1(time, z.top)
            d.accentTopExt.set_xy2(extTime, z.top)
            d.accentTopExt.set_color(color.new(css, 85))
        else
            d.accentTop.set_xy1(z.left_time, z.top)
            d.accentTop.set_xy2(time, z.top)
            d.accentTop.set_color(accentCol)
            d.accentTopExt.set_xy1(time, z.top)
            d.accentTopExt.set_xy2(extTime, z.top)
            d.accentTopExt.set_color(color.new(css, 40))
            d.accentBot.set_xy1(z.left_time, z.btm)
            d.accentBot.set_xy2(time, z.btm)
            d.accentBot.set_color(color.new(css, 75))
            d.accentBotExt.set_xy1(time, z.btm)
            d.accentBotExt.set_xy2(extTime, z.btm)
            d.accentBotExt.set_color(color.new(css, 85))

        // ——— Midline ———
        d.mL.set_xy1(z.left_time, z.avg)
        d.mL.set_xy2(time, z.avg)
        d.mL.set_color(color.new(css, 60))

        // ——— Label ———
        if i_showLabel
            string dom_txt = z.is_bull ? str.tostring(math.round(z.buy_pct, 1)) + "% BUY" :
str.tostring(math.round(z.sell_pct, 1)) + "% SELL"
            d.eOB.set_text(dom_txt)
            d.eOB.set_text_size(size.auto)
            d.eOB.set_text_halign(text.align_left)
            d.eOB.set_text_color(color.new(css, 20))
```

```
if i_show
    f_draw_all(bull_zones, bull_draws, i_bullCss, "bull")
    f_draw_all(bear_zones, bear_draws, i_bearCss, "bear")

// ┌──────────────────────────────────────────────────────────── BOSWaves — Alerts
──────────────────────────────────────────────────────────────────────────┐
//
 └──────────────────────────────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────────────────┘

alertcondition(fire_bull, title = "New Demand Stack", message = "Stacked Imbalance Demand |
{{exchange}}:{{ticker}}")
alertcondition(fire_bear, title = "New Supply Stack", message = "Stacked Imbalance Supply |
{{exchange}}:{{ticker}}")
```